

MATLAB REFERENCE FOR ENGINEERING

Public Posting Note

This reference uses general MATLAB commands. Some sections, such as Image Processing, Symbolic Math, and Parallel Computing, may require additional MATLAB toolboxes. Older commands such as `xlsread` and `csvread` are included for recognition, but modern functions such as `readmatrix`, `readtable`, and `readcell` are preferred.

WORKSPACE COMMANDS

```
format short      % Short numeric display
format long       % Long numeric display
disp(x)          % Display variable
fprintf()         % Formatted output
input()           % User input

clc              % Clear command window
clear            % Clear variables
clear all        % Clear all variables/functions
close all        % Close figures
who              % Show variable names
whos             % Show detailed variables
pwd              % Current folder
cd foldername    % Change folder
help plot        % Help for command

%              % Comment
;              % Suppress output
...           % Line continuation
'             % Transpose
end           % End loop/function
```

VARIABLES

```
x = 10;
y = 5;
name = 'MATLAB';
flag = true;

Rules:
• Start with a letter
• No spaces
• Case-sensitive
```

COMMON PLACEHOLDERS

```
%s      % string (text)
%c      % single character
%d      % integer (whole number)
%i      % integer
%f      % floating-point (decimal)
%.2f    % decimal with 2 digits after decimal
%e      % scientific notation
%g      % shortest format between %f and %e
%x      % hexadecimal
%o      % octal
%%      % print the percent sign itself
\n      % new line
```

```
% ===== Simple Examples =====
fprintf('Value = %d\n', x)
```

```
% ===== Common Format Specifiers
```

```
fprintf('%s\n', 'Hello')
```

```
fprintf('%d\n', 25)
```

```
fprintf('%f\n', 3.14159)
```

```
fprintf('Pi = %.2f\n', 3.14159)
```

```
fprintf('Success = 90%%\n')
```

IMPORTING DATA FROM FILES

```
% Numeric data import
readmatrix('data.xlsx')    % Read Excel/CSV/TXT as matrix

% Table import (mixed text + numbers)
readtable('data.xlsx')     % Read data as table

% Cell import (flexible mixed content)
readcell('data.xlsx')      % Read data as cell array

% Older Excel function
xlsread('data.xlsx')       % Old method for Excel files

% Text file import
load('data.txt')           % Load simple text/numeric file

% MAT-file import
load('data.mat')           % Load MATLAB saved variables

% CSV file import
csvread('data.csv')        % Older CSV function
readmatrix('data.csv')     % Modern CSV reading

% Text scanning
fopen('file.txt')          % Open text file
fscanf(fid,'%f')           % Read formatted data
textscan(fid,'%f %s')      % Advanced text reading
fclose(fid)                % Close file

% User file selection
uigetfile('*.xlsx')        % Open file browser

% Full path handling
fullfile(path,file)        % Create full file path
```

Most Recommended Modern Functions:

```
readmatrix()
readtable()
readcell()
```

STRINGS & CHARACTER ARRAYS

```
str = 'MATLAB';
length(str)          % Number of characters
upper(str)            % Convert to uppercase
lower(str)            % Convert to lowercase
strcmp(str,'MATLAB')  % Compare two strings
strcat('Hello',' World') % Join strings
num2str(123)          % Number to text
str2double('45.6')    % Text to number
```

Modern String Functions (R2016b+):

```
str = "MATLAB";      % Double quotes = string
split("Hello World") % Split into tokens
join(["a","b","c"],"-") % Join with delimiter
contains("MATLAB","TL") % Check if substring exists
replace("Hello","He","Je") % Replace text
```

Example:

```
name = 'student';
newName = upper(name)
```

ARITHMETIC OPERATIONS

```
a = 3 + 5;      % Addition
a = 3 - 5;      % Subtraction
a = 3 * 5;      % Multiplication
a = 3 / 5;      % Division
a = 3 ^ 5;      % Power
a = log(3);     % Natural logarithm
a = log10(3);   % Common logarithm
a = sqrt(9);    % Square root
```

SCRIPTS VS FUNCTIONS

```
% Script file: runs commands directly
% Example file name: myScript.m
x = 5;
y = x^2;
disp(y)
```

```
% Function file: takes inputs and returns outputs
% Example file name: squareNumber.m
function y = squareNumber(x)
    y = x^2;
end
```

Important: A function file name should match the function name.

FUNCTIONS

```
function [out1, out2] = function_name(input1, input2)
    % Body here
    out1 = input1 + input2;
    out2 = input1 - input2;
end
```

```
function [add, div] = MathOP(a,b,c,d)
    add = a + b;
    div = c / d;
end
```

Call Function

```
[x,y] = MathOP(3,4,10,2)
```

Important:

File name must be:
MathOP.m

VECTORS

Row Vector

```
v = [1 2 3 4 5]
```

Column Vector

```
v = [1;2;3;4;5]
```

MATRIX

Matrix Operations

```
A = [1; 2; 3];      % Column vector
A = [1 2 3];        % Row vector
A = [1, 2, 3];      % Row vector

A = [1 2 3; 4 5 6; 7 8 9]; % Matrix

A(1,1)              % Fetch single element
A(1)                % Linear indexing
A(:,1)              % Fetch column
A(1,:)              % Fetch row
A(1:2,2:3)          % Fetch part of matrix

size(A)             % Matrix size
eye(3)              % Identity matrix
ones(3)             % Matrix of ones
zeros(3)            % Matrix of zeros
magic(3)            % Magic matrix
```

LOGICAL OPERATIONS

```
a > b      % Greater than
a < b      % Less than
a >= b     % Greater or equal
a <= b     % Less or equal
a == b     % Equal
a ~= b     % Not equal
a && b      % AND for scalar values
a || b     % OR for scalar values
A & B      % Element-wise AND for arrays
A | B      % Element-wise OR for arrays
~a         % NOT
any(A)     % True if any value is true
all(A)     % True if all values are true
find(A > 5) % Find indexes
```

Example:

```
x = [2 6 9 1];
index = find(x > 5)
```

SET OPERATIONS

```
A = [1 2 3 4];
B = [3 4 5 6];
```

```
union(A,B)      % Combine unique values
intersect(A,B)   % Common values
setdiff(A,B)     % Values in A not in B
setxor(A,B)      % Values not shared
ismember(3,A)    % Check membership
unique([1 2 2 3]) % Remove duplicates
```

Example:

```
common = intersect(A,B)
```

POLYNOMIALS & INTERPOLATION

```
p = [1 -3 2];      % x^2 - 3x + 2
roots(p)            % Polynomial roots
poly([1 2])         % Polynomial from roots
polyval(p,5)        % Evaluate polynomial at x=5
conv([1 2],[1 3])   % Multiply polynomials
```

```
x = [1 2 3];
y = [2 4 8];
interp1(x,y,2.5)    % Interpolation
```

Example:

```
p = [1 -3 2];
r = roots(p)
```

IMAGE PROCESSING

Read Display and Save Images

```
impath = 'image.jpg'; % Image path
img = imread(impath); % Read image
imshow(img);          % Show image
imwrite(img,'new_image.jpg'); % Save image

figure;               % New figure window
imshow(img);          % Show image in figure
title('Original Image'); % Add title
```

MATLAB Figure Files

```
openfig('MoBessel.fig'); % Open MATLAB .fig file
open('MoBessel.fig');    % Open MATLAB figure/file
saveas(gcf,'MoBessel.png'); % Save current fig as image
```

Image Dimensions

Linear Algebra Commands

```
det(A)           % Determinant
inv(A)           % Inverse matrix
rank(A)          % Matrix rank
eig(A)           % Eigenvalues
transpose(A)     % Transpose
A'
```

Matrix Arithmetic

Example Matrices

```
A = [2 4 6; 8 10 12; 14 16 18] % First matrix
B = [1 3 5; 7 9 11; 13 15 17] % Second matrix
```

```
A + 2      % Add scalar to matrix
A + B      % Add two matrices
A - 2      % Subtract scalar from matrix
A - B      % Subtract two matrices
A * 2      % Scalar multiplication
A * B      % Matrix multiplication
A .* B     % Element-by-element multiplication
A ./ B     % Element-by-element division
A .^ 2     % Element-by-element power
```

Sparse Matrices

```
A = [1 0 0; 0 5 0; 0 0 9];
S = sparse(A) % Convert to sparse matrix
full(S)       % Convert back to full matrix
spy(S)        % Show nonzero structure
nnz(S)        % Number of nonzero elements
```

Example:

```
S = sparse(A);
numberNonZero = nnz(S)
```

INDEXING

```
A(2,3) % Row 2 column 3
A(2,:) % Full row 2
A(:,3) % Full column 3
```

SYMBOLIC MATH

Differential Equations

```
ode45() % Solve differential equations
```

```
syms x
```

```
diff(x^2) % Derivative
int(x^2)  % Integral
solve(x^2 - 4 == 0) % Solve equation
limit(sin(x)/x,x,0) % Limit
subs(x^2,x,5) % Substitute value
simplify((x^2-1)/(x-1)) % Simplify expression
```

Example:

```
syms x
answer = solve(x^2 - 9 == 0)
```

PARALLEL COMPUTING

```
parpool(4) % Start 4 workers
result = zeros(1,5);
parfor i = 1:5
    result(i) = i^2; % Each iteration runs in parallel
end
delete(gcf) % Close pool
```

Example:

```
dimensions = size(img); % Get image dimensions
nr = dimensions(1); % Number of rows
nc = dimensions(2); % Number of columns
channels = dimensions(3); % Number of color channels
```

Image Type Conversion

```
gray = rgb2gray(img); % RGB to grayscale
bw = imbinarize(gray); % Convert grayscale to binary image
img_double = im2double(img); % Convert image to double
img_uint8 = im2uint8(img_double); % Convert image to uint8
indexed = gray2ind(gray); % Grayscale to indexed image
gray2 = mat2gray(A); % Matrix to grayscale image
[r,g,b] = imsplit(img); % Split RGB channels
```

Resize Rotate and Crop

```
small = imresize(img,0.5); % Resize image to 50 percent
rot = imrotate(img,45); % Rotate image by 45 degrees
crop = imcrop(img); % Crop image manually
```

Display Multiple Images

```
subplot(1,2,1), imshow(img); % First image
title('Original Image');
```

```
subplot(1,2,2), imshow(gray); % Second image
title('Grayscale Image');
```

Histogram and Contrast

```
imhist(gray); % Show grayscale histogram
adjusted = imadjust(gray); % Improve contrast
equalized = histeq(gray); % Histogram equalization
```

Filtering

```
smooth = imgaussfilt(gray,2); % Gaussian smoothing filter
medianImg = medfilt2(gray); % Median filter
sharp = imsharpen(gray); % Sharpen image
```

Thresholding

```
level = graythresh(gray); % Automatic threshold level
bw = imbinarize(gray,level); % Convert to binary using threshold
```

Noise

```
noisy = imnoise(gray,'salt & pepper'); % Add salt and pepper noise
noisy2 = imnoise(gray,'gaussian'); % Add Gaussian noise
```

Edge Detection

```
edges1 = edge(gray,'Sobel'); % Sobel edge detection
edges2 = edge(gray,'Prewitt'); % Prewitt edge detection
edges3 = edge(gray,'Canny'); % Canny edge detection
```

Morphological Operations

```
se = strel('disk',5); % Structuring element
dilated = imdilate(bw,se); % Dilate white regions
eroded = imerode(bw,se); % Erode white regions
opened = imopen(bw,se); % Remove small noise
closed = imclose(bw,se); % Fill small gaps
filled = imfill(bw,'holes'); % Fill holes
cleaned = bwareaopen(bw,50); % Remove small objects
```

Object Detection and Measurement

```
cc = bwconncomp(bw); % Find connected components
stats = regionprops(bw,'Area','Centroid'); % Measure object properties
labeled = bwlabel(bw); % Label connected objects
```

Color Processing

```
hsvImg = rgb2hsv(img); % RGB to HSV
rgbImg = hsv2rgb(hsvImg); % HSV to RGB
labImg = rgb2lab(img); % RGB to Lab color space
```

```
A = zeros(1,5);
parfor i = 1:5
    A(i) = i^2;
end
```

LOOPS AND CONDITIONS

If	if condition % code end	% checks once
For	for i = 1:n % code end	% repeats through range
While	while condition % code end	% repeats while true
Function	function y=fun(x) % code end	% reusable code
Switch	switch value case 1 % code otherwise % code end	% chooses cases
Try-Catch	try % code catch % code end	% handles errors
Break	break	% exits loop
Continue	continue	% skips one step
Anonymous	f = @(x) x^2;	% one-line function

GRAPHS AND PLOTS

Advanced Plotting

```
fplot()          % Function plotting
mesh()           % 3D mesh plot
surf()           % Surface plot
contour()        % Contour plot

% Data
X = linspace(-2*pi,2*pi,100);
Y1 = sin(X);
Y2 = cos(X);

% Line plot
plot(X,Y1,'r','LineWidth',2)
hold on
plot(X,Y2,'b--','LineWidth',2)

% Labels, title, legend
xlabel('-2\pi < X < 2\pi','FontSize',12)
ylabel('Amplitude','FontSize',12)
title('Sine and Cosine Waves','FontSize',14)
legend({'sin(x)','cos(x)'},'Location','northeast')

% Grid and axis font
grid on
set(gca,'FontSize',11)

% Bar chart
bar([100 200 300 400],[3 6 9 12])

% Histogram of grayscale image
imhist(imgray)
```

Geometric Transformations

```
flipped1 = flipud(img);          % Flip image up-down
flipped2 = fliplr(img);          % Flip image left-right
translated = imtranslate(img,[20 30]); % Shift image by x=20, y=30
```

Fourier Image Processing

```
F = fft2(gray);          % 2D Fourier transform
Fshift = fftshift(F);     % Shift zero frequency to center
mag = log(1 + abs(Fshift)); % Fourier magnitude image
imshow(mag,[]);           % Display magnitude image
```

Basic Image Information

```
info = imfinfo('image.jpg'); % Get image file information
class(img);                   % Image data type
```

Example:

```
img = imread('image.jpg'); % Read image
gray = rgb2gray(img);      % Convert to grayscale
imshow(gray);              % Show grayscale image
imwrite(gray,'gray_image.jpg'); % Save grayscale image
```

AUDIO & SIGNAL PROCESSING

```
[y,Fs] = audioread('sound.wav'); % Read audio
sound(y,Fs)                        % Play audio
plot(y)                            % Plot signal
Y = fft(y);                        % Frequency domain
y2 = ifft(Y);                      % Inverse FFT
```

```
t = 0:0.001:1;                    % Time vector
x = sin(2*pi*10*t);               % Signal generation
plot(t,x)                         % Plot signal
```

Example:

```
t = 0:0.001:1;                    % Time vector
x = sin(2*pi*5*t);               % Sine wave
plot(t,x)                        % Plot signal
```

DESKTOP & GUI

```
uigetfile          % Open file selection window
uiputfile          % Save file selection window
msgbox('Hello')    % Message box
inputdlg('Enter value') % Input dialog
menu('Choose','Yes','No') % Menu dialog
waitbar(0.5,'Loading') % Progress bar
figure             % Create figure window
uitable            % Create table in figure
uicontrol          % Create GUI control
```

Example:

```
answer = inputdlg('Enter your name:');
msgbox(['Hello ', answer{1}])
```

BITWISE LOGICAL OPERATIONS

```
a = bitor(3,5); % Bitwise OR
a = bitand(3,5); % Bitwise AND
a = bitcmp(3); % Bitwise NOT
a = bitxor(3,5); % Bitwise XOR
```

MINIMUM, MAXIMUM, AND MEDIAN

```
min_value = min(vector); % Minimum of vector
min_value = min(mat(:)); % Minimum of matrix

max_value = max(vector); % Maximum of vector
max_value = max(mat(:)); % Maximum of matrix

median_value = median(vector); % Median of vector
median_value = median(mat(:)); % Median of matrix
```

```
% Full customization example
plot(X,Y1,...
    'Color',[0 0.5 0],...
    'LineStyle','--',...
    'LineWidth',2,...
    'Marker','o',...
    'MarkerSize',8,...
    'MarkerFaceColor','y')

% Common line styles:
% '-' solid, '--' dashed, ':' dotted, '-.' dash-dot

% Common markers:
% 'o' circle, 'x' cross, '*' star, 's' square

% Combine style, marker, and color
plot(X,Y1,'--or',X,Y2,'--xb')
```

BUILT IN FUNCTIONS

```
% Basic Functions
length(x)      % Length of vector or string
size(A)        % Size of matrix
sum(x)         % Sum of elements
mean(x)        % Average value
max(x)         % Maximum value
min(x)         % Minimum value
sqrt(x)        % Square root
abs(x)         % Absolute value
round(x)       % Round number
floor(x)       % Round down
ceil(x)        % Round up
sort(x)        % Sort values
unique(x)      % Remove duplicates
find(x > 5)    % Find index locations
numel(x)       % Number of elements
isempty(x)     % Check if empty
isequal(A,B)   % Compare variables

% Mathematical Functions
mod(10,3)      % Remainder after division
factorial(5)   % Factorial value
log(x)         % Natural logarithm
log10(x)       % Base-10 logarithm
exp(x)         % Exponential function

% Trigonometric Functions
sin(x)         % Sine
cos(x)         % Cosine
tan(x)         % Tangent
asin(x)        % Inverse sine
acos(x)        % Inverse cosine
atan(x)        % Inverse tangent

% Bessel Functions
besselj(n,x)   % Bessel function of first kind
bessely(n,x)   % Bessel function of second kind
besseli(n,x)   % Modified Bessel function first kind
besselk(n,x)   % Modified Bessel function second kind

% Special Functions
gamma(x)       % Gamma function
beta(x,y)     % Beta function
erf(x)         % Error function
sinc(x)        % Sinc function
```

DATE & TIME

```
date           % Current date as text
clock          % Current date/time vector
now            % Current date/time number
datetime('now') % Current date/time object
year(datetime('now')) % Current year
month(datetime('now')) % Current month
datestr(now)   % Convert date to string
tic; pause(1); toc % Measure elapsed time
```

FILE INPUT / OUTPUT (I/O)

Quick Examples

```
save data.mat           % Save workspace variables
save('myfile.mat')     % Save MAT-file

load data.mat           % Load workspace variables
load('myfile.mat')      % Load MAT-file

readmatrix('data.xlsx') % Read numeric data
readtable('data.csv')   % Read table data
readcell('data.xlsx')   % Read mixed data

writematrix(A,'output.csv') % Write matrix
writetable(T,'table.xlsx') % Write table

saveas(gcf,'figure.png') % Save figure

fid = fopen('file.txt','w') % Open text file
fprintf(fid,'Hello MATLAB') % Write text
fclose(fid)                 % Close file

pwd           % Current folder
cd foldername % Change folder
dir           % List folder files

path           % Show MATLAB path
addpath('C:\MATLAB\MyFiles') % Add folder to path
rmpath('C:\MATLAB\OldFiles') % Remove folder from path

uigetfile('*.xlsx') % File browser window
uiputfile('*.mat')  % Save file browser

fullfile(path,file) % Full file path
```

USER INTERACTION

```
input()      % user input from command window
disp()       % display output
fprintf()    % formatted output
sprintf()    % create formatted text string
num2str()    % convert number to string
str2double() % convert string to number

msgbox()      % message box popup
inputdlg()    % input dialog box
menu()        % menu selection dialog
questdlg()    % question dialog box
warndlg()     % warning dialog box
errordlg()    % error dialog box
helpdlg()     % help dialog box
listdlg()     % list selection dialog

uigetfile()   % open file selection window
uiputfile()   % save file selection window
uigetdir()    % select folder window

waitbar()     % progress/loading bar
pause()       % pause execution
keyboard      % pause & enter debug mode
uiwait()      % wait for user action
uiresume()    % resume after uiwait
beep          % sound notification

clc          % clear command window
clear        % clear workspace variables
close all    % close all figure windows
format       % control number display format
```

```
datestr() % convert date to text
clock     % current date and time

tic % start timer
toc % stop timer and show elapsed time
```

DATA ANALYSIS

Random Numbers

```
rand(3)           % Random numbers
randi(10)          % Random integers
randn(3)           % Normal distribution

x = [10 20 30 40 50];

max(x)            % Maximum
min(x)            % Minimum
mean(x)           % Average
median(x)         % Middle value
mode(x)           % Most frequent value
std(x)            % Standard deviation
var(x)            % Variance
sum(x)            % Sum
sort(x)           % Sort values
cumsum(x)         % Cumulative sum
diff(x)           % Differences between elements
corrcoef(x,y)% Correlation coefficient
histogram(x) % Histogram plot
```

Example:

```
grades = [80 90 75 88];
averageGrade = mean(grades)
```

DEBUGGING COMMANDS

```
dbstop if error      % Stop execution when an error occurs
dbstop if warning    % Stop execution when a warning occurs
dbstop in file at N  % Stop at line N in a specific file

dbclear all          % Remove all breakpoints
dbclear if error      % Remove stop on error
dbclear if warning    % Remove stop on warning

dbstep               % Execute next line during debugging
dbcont               % Continue execution after pause
dbquit               % Exit debug mode completely

keyboard             % Pause execution manually

disp(variable)       % Display variable value
fprintf()             % Print formatted debug information

whos                  % Show all variables in workspace
who                   % Show variable names only
size(variable)        % Show variable dimensions
class(variable)       % Show variable data type

error('message')      % Generate custom error message
warning('message')    % Generate warning message
assert(condition)     % Stop if condition is false

try                  % Begin error handling block
catch                % Catch and handle errors
rethrow(ME)          % Re-throw caught error
ME.exception          % Error object information
```